

Technical PRD — AI Footage Discovery & Relevance Engine

1. Product Vision

Build an AI-powered footage retrieval engine that helps video editors rapidly find relevant stock footage and YouTube clips for narration scripts.

The system should:

- understand narration semantically
- infer visual intent
- retrieve candidate footage
- identify exact useful timestamps
- rank clips by cinematic usefulness
- continuously improve through indexing

2. Primary Users

- YouTube editors
- Documentary creators
- Short-form content creators
- Agencies
- AI video generation pipelines
- Faceless YouTube automation channels

3. Core User Flow

1. User uploads script
2. System chunks script semantically
3. System generates visual intent
4. System searches Pexels and indexed YouTube database
5. System ranks footage relevance
6. System returns best clips, timestamps, and scores
7. User selects clips
8. Export timeline metadata

4. Functional Requirements

Key modules:

- Script ingestion
- Semantic chunking
- Visual intent generation
- Stock footage search
- YouTube ingestion
- Video segmentation
- Frame sampling
- Embedding generation
- Vector database
- Retrieval engine
- LLM reranking
- Cinematic quality scoring

5. Recommended Architecture

- SCRIPT
 - semantic chunks
 - visual intent generation

- embedding search

YOUTUBE VIDEO

- transcript
- timestamp segmentation
- sampled frames
- CLIP embeddings
- metadata indexing

MATCHING ENGINE

- embedding similarity
- LLM reranking
- timestamp extraction
- usefulness scoring

6. Recommended Tech Stack

Backend: Python + FastAPI
Async Jobs: Celery / RQ
Video Processing: ffmpeg
YouTube Extraction: yt-dlp
Embeddings: sentence-transformers
Vision Embeddings: CLIP
Vector Search: FAISS
LLM APIs: Groq / OpenRouter
Database: PostgreSQL
Frontend: React / Next.js

7. MVP Scope

MUST HAVE:

- transcript chunking
- embeddings
- semantic retrieval
- timestamp recommendations
- LLM reranking

SHOULD HAVE:

- CLIP frame matching
- cinematic scoring

LATER:

- automatic timeline generation
- Premiere plugin
- AI rough cuts

8. Cost Strategy

Use local models whenever possible:

- CLIP locally
- sentence-transformers locally
- FAISS locally
- ffmpeg locally

Use free API tiers:

- Groq
- OpenRouter

Avoid early dependency on:

- GPT-4 Vision
- expensive vector databases
- full-video multimodal analysis

9. Key Technical Insight

The system should not attempt full video understanding.

The optimal architecture is:

lightweight indexing
+ embedding retrieval
+ targeted LLM judgment

instead of expensive full-video multimodal analysis.

Generated for Erfan Salehi — AI Footage Assistant Technical PRD